ENGINEERING & ARCHITECTURE SPECIFICATION • SPOKEN TUTORIAL, IIT BOMBAY

Spoken Social — Technical & Architecture Brief

System Architecture, Data Invariants, and Platform Engineering for Scale

System Vital Statistics (Production Audit — August 2026)

Active Deployment: Single cloud VM running 11 Docker Compose services.

Workload: 5,446 students, 173 colleges, 4,633 course enrollments across 2 cohorts.

API & Database: FastAPI (async end-to-end), PostgreSQL 16 (27 tables, 30 indexes, 19 Alembic migrations).

Background Jobs: arq worker on Redis 7 (15-min forum XP sweep, nightly 02:00 IST invariant audits).

Observability: Prometheus, Loki, Alloy, node-exporter, cAdvisor, and a 12-tool Model Context Protocol (MCP) server.

1. Engineering Thesis & Core Philosophy

The central engineering principle of Spoken Social is correctness at scale. Experience points (XP) and student progress behave like accounting ledger state—written concurrently by user clicks, external forum webhooks, batch administrative actions, and legacy CSV data imports.

To guarantee absolute consistency without sacrificing high concurrency, the platform enforces three foundational invariants: (1) a single-writer idempotent XP engine, (2) an append-only event journal, and (3) automated nightly machine-checked SQL integrity invariants.

Furthermore, the architecture embraces 'Fat Database, Thin Application': aggregation logic runs entirely inside PostgreSQL via conditional grouping, keeping query counts constant regardless of student or college volume.

2. System Architecture & Service Topology

Component	Technology	Role & Engineering Responsibility
API Server	FastAPI 0.133 (Python 3.10+)	Fully asynchronous ASGI service. 36 REST endpoints across 10 routers. Connection pooling via asyncpg.
Database	PostgreSQL 16	27 relational tables, 30 declared indexes including covering B-trees with INCLUDE clauses. 19 Alembic migrations.
Cache / Queue	Redis 7	Arq task queue, fail-open TTL caching on analytics endpoints, worker-to-backend handoff channel.
Background Worker	arq (Python async)	Runs out-of-process background jobs: 15-minute forum verification sweeps and nightly invariant checks.
Frontend SPA	React 19 + TypeScript (strict)	Vite 7, Tailwind CSS 4, Zustand store, TanStack Query (optimistic mutations), Recharts.

Component	Technology	Role & Engineering Responsibility
Observability	Prometheus + Loki + Alloy	System metrics, container telemetry, structured JSON log shipping, cardinality-tuned scrapers.
AI Operations	Custom MCP Server (Python)	12-tool read-only Model Context Protocol server exposing DB queries, metrics, and logs safely to LLM agents.

3. Core Architectural Highlights

A. Single-Writer XP Engine & Event Journaling

- **Deterministic Single-Writer:** UserTutorial.xp_earned is NEVER incremented in place. Instead, services/xp.py::recompute_tutorial_xp computes the exact sum of XP_MAP over the row's boolean flags (watched: 10, practiced: 10, assignment_done: 5, forum_asked: 5, forum_answered: 10). It is strictly idempotent and returns the net delta to apply to user.xp_total.
- **Append-Only Journal:** Every XP change appends an immutable record to xp_events containing user_id, amount, source_type, reference_id, and timestamp. Double-counting is structurally impossible; duplicate webhook deliveries produce a 0 delta.

B. Nightly Machine-Checked SQL Invariants

- **Dynamically Generated Rules:** Seven mathematical consistency checks are generated directly from the Python XP rules, guaranteeing assertions never become stale.
- **Nightly Worker Sweep:** Executed nightly at 02:00 IST by the arq worker. The worker verifies that: (1) User XP matches the sum of tutorial flags, (2) User XP matches the sum of journaled events, (3) User level matches threshold rules, and (4) Forum XP corresponds to surviving posts.
- **Metric Integration:** Violations are exported to Prometheus gauges (spoken_social_xp_invariant_violations) with student UUIDs, alerting operators immediately.

C. Constant-Cost Institutional Analytics (Fat Database, Thin Application)

- **Constant Query Count:** Coordinator and admin analytics aggregate student cohorts using SQL CASE WHEN constructs and grouped subqueries. Fetching the multi-cohort breakdown for 170+ colleges takes exactly 3 SQL queries.
- **Single Admin Query:** The entire platform-wide admin overview runs in a single aggregate query. Cohorts are never loaded into Python memory, avoiding N+1 memory leaks.

D. Sub-Millisecond Uncached Leaderboards

- **Native SQL Ranking:** Leaderboards use SQL RANK() OVER (ORDER BY xp_total DESC) directly against PostgreSQL composite descending indexes.
- **Uncached by Design:** Benchmark tests demonstrated a 0.11 ms read latency for top-100 queries. Consequently, Redis leaderboard caching was deliberately removed, eliminating cache-coherence race conditions without any performance penalty.

E. Anti-Abuse Forum XP Verification

- **24-Hour Hold Window:** Forum interactions arrive via signed webhooks. To prevent 'post-and-delete' farming, an arq cron job sweeps posts after a 24-hour verification window; XP is awarded only if the post remains active.

F. External Auth & Audited Impersonation

- **External Token Verification:** Zero local passwords. User identity is verified against Spoken Tutorial's credential API, issuing stateless JWTs (HS256, 24-hr expiry).
- **Full Audit Logging:** Admins can impersonate students or coordinators for support; the JWT embeds both real and effective identities, and every session is logged to impersonation_logs.

4. Database Architecture & Key Optimizations

Table Name	Schema & Constraints	Index Optimization
users	UUID primary key, foreign key to college_id. Tracks xp_total, level, and is_test_account (NOT NULL).	idx_users_college_xp (composite descending)
user_tutorials	Composite PK (user_id, tutorial_id). Stores flags: watched, practiced, assignment_done, etc.	idx_user_tutorials_tutorial_cover (covering with INCLUDE)
xp_events	Append-only ledger. Stores amount, source (tutorial/forum), reference_id, and created_at.	idx_xp_events_user_created (indexed for time-window queries)
batches & courses	Organizes programmes -> courses -> batches hierarchy. Supports multi-tenancy and archiving.	idx_batches_college_course, uq_batch_name_college
feedbacks	Stores student roadblocks when practice/assignment is marked false, with JSONB reasons.	idx_feedbacks_user_tutorial

Production Hardening Lesson: SQL Three-Valued Logic

A notable production bug occurred when querying leaderboards with `WHERE is_test_account != TRUE`. In SQL's three-valued logic, `NULL != TRUE` evaluates to `NULL` (falsy), silently hiding 2,441 students (77 of the top 100) from rankings. The schema was permanently hardened with a database migration enforcing `is_test_account NOT NULL DEFAULT FALSE` and `IS NOT TRUE` guards across all queries.

5. Observability & AI Operations (MCP)

Spoken Social implements comprehensive multi-tier monitoring: container metrics via cAdvisor, host stats via node-exporter, and Prometheus metric scrapers. Grafana Alloy ships structured logs to Loki.

In addition, an in-house 12-tool Model Context Protocol (MCP) server provides AI coding agents and automated assistants with structured, read-only operational capabilities:

- **Database Inspection Tools:** Read-only access to query students, colleges, XP logs, and database schema without write privileges.
- **Prometheus Tooling:** Direct access to Prometheus metric timeseries and query instant/range endpoints for performance triage.
- **Loki Log Inspection:** Safe Loki log stream queries with structured audit logging (tool call arguments are logged; sensitive user data is excluded).

6. Technical Roadmap for Scale (500+ Colleges / 200K+ Students)

- **1. Rate Limiting Middleware:** Implement sliding-window token bucket rate limiting in Redis on high-frequency endpoints (/api/tutorials/progress and webhook receivers).
- **2. Read Replicas & Connection Pooling:** Deploy PostgreSQL read replicas with PgBouncer connection pooling to offload analytical queries from the primary transactional instance.
- **3. CDN Asset Offloading:** Migrate static assets and frontend bundles to AWS S3/CloudFront or BunnyCDN, reducing origin load.
- **4. Social Feed Optimization:** Re-enable the social feed with cursor-based keyset pagination (id < cursor) replacing offset pagination.
- **5. CI/CD & Automated Regression Harness:** Automate staging deployments, database rollback testing, and frontend Playwright integration test suites.